Introduction to OpenGL

As a software interface for graphics hardware, OpenGL's main purpose is to render two- and three-dimensional objects into a frame buffer. These objects are described as sequences of vertexes (which define geometric objects) or pixels (which define images). OpenGL performs several processing steps on this data to convert it to pixels to form the final desired image in the frame buffer.

The following topics present a global view of how OpenGL works:

- Primitives and Commands discusses points, line segments, and polygons as the basic units of drawing; and the processing of commands.
- OpenGL Graphic Control describes which graphic operations OpenGL controls and which it does not control.
- Execution Model discusses the client-server model for interpreting OpenGL commands.
- Basic OpenGL Operation gives a high-level description of how OpenGL processes data and produces a corresponding image in the frame buffer.

Primitives and Commands

OpenGL draws primitives—points, line segments, or polygons—subject to several selectable modes. You can control modes independently of one another. That is, setting one mode doesn't affect whether other modes are set (although many modes may interact to determine what eventually ends up in the frame buffer). Primitives are specified, modes are set, and other OpenGL operations are described by issuing commands in the form of function calls.

Primitives are defined by a group of one or more *vertices.* A vertex defines a point, an endpoint of a line, or a corner of a polygon where two edges meet. Data (consisting of vertex coordinates, colors, normals, texture coordinates, and edge flags) is associated with a vertex, and each vertex and its associated data are processed independently, in order, and in the same way. The only exception to this rule is if the group of vertices must be clipped so that a particular primitive fits within a specified region. In this case, vertex data may be modified and new vertices created. The type of clipping depends on which primitive the group of vertices represents.

Commands are always processed in the order in which they are received, although there may be an indeterminate delay before a command takes effect. This means that each primitive is drawn completely before any subsequent command takes effect. It also means that state-querying commands return data that is consistent with complete execution of all previously issued OpenGL commands.

OpenGL Graphic Control

OpenGL provides you with fairly direct control over the fundamental operations of two- and three-dimensional graphics. This includes specification of such parameters as transformation matrices, lighting equation coefficients, antialiasing methods, and pixel update operators. However, it doesn't provide you with a means for describing or modeling complex geometric objects. Thus, the OpenGL commands you issue specify how a certain result should be produced (what procedure should be followed) rather than what exactly that result should look like. That is, OpenGL is fundamentally procedural rather than descriptive. Because of this procedural nature, it helps to know how OpenGL works—the order in which it carries out its operations, for example—to fully understand how to use it.

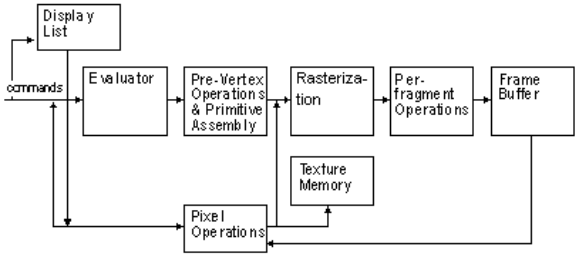
Execution Model

The model for interpretation of OpenGL commands is client-server. An application (the client) issues commands, which are interpreted and processed by OpenGL (the server). The server may or may not operate on the same computer as the client. In this sense, OpenGL is network-transparent. A server can maintain several GL contexts, each of which is an encapsulated GL state. A client can connect to any one of these contexts. The required network protocol can be implemented by augmenting an already existing protocol or by using an independent protocol. No OpenGL commands are provided for obtaining user input.

The effects of OpenGL commands on the frame buffer are ultimately controlled by the window system that allocates frame buffer resources. The window system determines which portions of the frame buffer OpenGL may access at any given time and communicates to OpenGL how those portions are structured. Therefore, there are no OpenGL commands to configure the frame buffer or initialize OpenGL. Frame buffer configuration is done outside of OpenGL in conjunction with the window system; OpenGL initialization takes place when the window system allocates a window for OpenGL rendering.

Basic OpenGL Operation

The figure shown below gives an abstract, high-level block diagram of how OpenGL processes data. In the diagram, commands enter from the left and proceed through what can be thought of as a processing pipeline. Some commands specify geometric objects to be drawn, and others control how the objects are handled during the various processing stages.



As shown in the diagram's first block, rather than having all commands proceed immediately through the pipeline, you can choose to accumulate some of them in a display list for processing later.

The evaluator stage of processing provides an efficient means for approximating curve and surface geometry by evaluating polynomial commands of input values. During the next stage, per-vertex operations and primitive assembly, OpenGL processes geometric primitives—points, line segments, and polygons, all of which are described by vertices. Vertices are transformed and lit, and primitives are clipped to the viewport in preparation for the next stage.

Rasterization produces a series of frame buffer addresses and associated values using a two-dimensional description of a point, line segment, or polygon. Each fragment so produced is fed into the last stage, per-fragment operations, which performs the final operations on the data before it's stored as pixels in the frame buffer. These operations include conditional updates to the frame buffer based on incoming and previously stored z-values (for z-buffering) and blending of incoming pixel colors with stored colors, as well as masking and other logical operations on pixel values.

Input data can be in the form of pixels rather than vertices. Such data, which might describe an image for use in texture mapping, skips the first stage of processing described above and instead is processed as pixels, in the pixel operations stage. The result of this stage is either

stored as texture memory, for use in the rasterization stage, or rasterized and the resulting fragments merged into the frame buffer just as if they were generated from geometric data.

All elements of an OpenGL state, including the contents of the texture memory and even of the frame buffer, can be obtained by an OpenGL application.

OpenGL Processing Pipeline

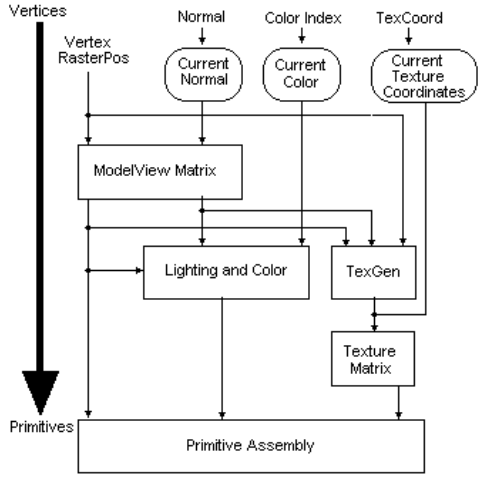
Many OpenGL commands pertain specifically to drawing objects such as points, lines, polygons, and bitmaps. Other commands control the way that some of this drawing occurs (such as those that enable antialiasing or texturing). Still other commands are specifically concerned with frame buffer manipulation. This section describes how all the OpenGL commands work together to create the OpenGL processing pipeline.

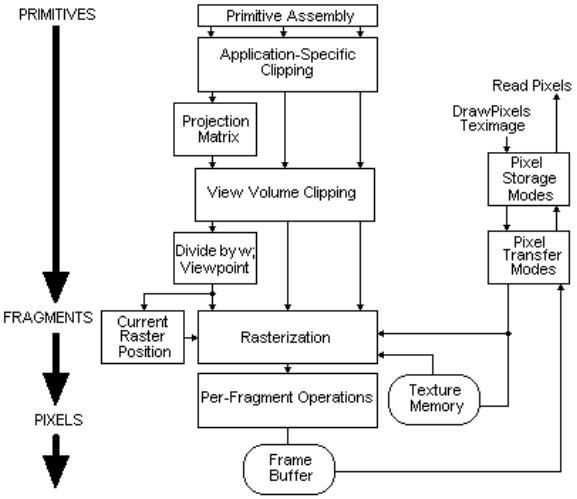
This section takes a closer look at the stages in which data is actually processed, and ties these stages to OpenGL commands. Scroll to the end of this topic to see a more detailed block diagram of the OpenGL processing pipeline.

For most of the pipeline, you can see three vertical arrows between the major stages. These arrows represent vertices and the two primary types of data that can be associated with vertices: color values and texture coordinates. Also note that vertices are assembled into primitives, then to fragments, and finally to pixels in the frame buffer. This progression is discussed in more detail in **Vertices**, **Primitives**, **Fragments**, and **Pixels**.

As you continue reading, be aware that we've taken some liberties with command names. Many OpenGL commands are simple variations of each other, differing mostly in the data type of arguments. Some commands differ in the number of related arguments and whether those arguments can be specified as a vector or whether they must be specified separately in a list. For example, if you use the **glVertex2f** command, you need to supply *x* and *y* coordinates as 32-bit floating-point numbers; with **glVertex3sv**, you must supply an array of three short (16-bit) integer values for *x*, *y*, and *z*. For simplicity, only the base name of the command is used in the topics that follow, and an asterisk is included to indicate that there may be more to the actual command name than is shown. For example, **glVertex** stands for all variations of the command you use to specify vertices.

Also keep in mind that the effect of an OpenGL command may vary depending on whether certain modes are enabled. For example, you need to enable lighting if the lighting-related commands are to produce a properly lit object. To enable a particular mode, use the **glEnable** command and supply the appropriate constant to identify the mode (for example, GL_LIGHTING). Refer to **glEnable** for a complete list of the modes that can be enabled. Modes are disabled with **glDisable**.





Vertices

This topic relates the OpenGL commands that perform per-vertex operations to the processing stages shown in **OpenGL Processing Pipeline**.

Input Data

You must provide several types of input data to the OpenGL pipeline:

- Vertices—Vertices describe the shape of the desired geometric object. To specify vertices,

use **glVertex** commands in conjunction with **glBegin** and **glEnd** to create a point, line, or polygon. You can also use **glRect** to describe an entire rectangle at once.

- Edge flag—By default, all edges of polygons are boundary edges. Use the **glEdgeFlag** command to explicitly set the edge flag.
- Current raster position—Specified with **glRasterPos**, the current raster position is used to determine raster coordinates for pixel and bitmap drawing operations.
- Current normal—A normal vector associated with a particular vertex determines how a surface at that vertex is oriented in three-dimensional space; this in turn affects how much light that particular vertex receives. Use **glNormal** to specify a normal vector.
- Current color—The color of a vertex, together with the lighting conditions, determine the final, lit color. Color is specified with **glColor** if in RGBA mode or with **glIndex** if in color index mode.
- Current texture coordinates—Specified with **glTexCoord**, texture coordinates determine the location in a texture map that should be associated with a vertex of an object.
- When **glVertex** is called, the resulting vertex inherits the current edge flag, normal, color, and texture coordinates. Therefore, **glEdgeFlag**, **glNormal**, **glColor**, and **glTexCoord** must be called before **glVertex** if they are to affect the resulting vertex.

Matrix Transformations

Vertices and normals are transformed by the modelview and projection matrices before they're used to produce an image in the frame buffer. You can use commands such as **glMatrixMode**, **glMultMatrix**, **glRotate**, **glTranslate**, and **glScale** to compose the desired transformations, or you can directly specify matrices with **glLoadMatrix** and **glLoadIdentity**. Use **glPushMatrix** and **glPopMatrix** to save and restore modelview and projection matrices on their respective stacks.

Lighting and Coloring

In addition to specifying colors and normal vectors, you may define the desired lighting conditions with **glLight** and **glLightModel**, and the desired material properties with **glMaterial**. Related commands for controlling how lighting calculations are performed include **glShadeModel**, **glFrontFace**, and **glColorMaterial**.

Generating Texture Coordinates

Rather than explicitly supplying texture coordinates, you can have OpenGL generate them as a function of other vertex data. This is what the **glTexGen** command does. After the texture coordinates have been specified or generated, they are transformed by the texture matrix. This matrix is controlled with the same commands for matrix transformations.

Primitive Assembly

Once all necessary calculations have been performed, vertices are assembled into **primitives**—points, line segments, or polygons—together with the relevant edge flag, color, and texture information for each vertex.

Vertexes Reference

glBegin
glColor
glColorMaterial
glEdgeFlag
glEnd
glFrontFace
glIndex
glLight
glLightModel

glLoadIdentity
glLoadMatrix
glMaterial
glMatrixMode
glMultMatrix
glNormal
glPopMatrix
glPushMatrix
glRasterPos
glRect
glRotate
glScale
glShadeModel
glTexCoord
glTexGen
glTranslate
glVertex

Primitives

Primitives are converted to pixel fragments in several steps: primitives are clipped appropriately, whatever corresponding adjustments are necessary are made to the color and texture data, and the relevant coordinates are transformed to window coordinates. Finally, rasterization converts the clipped primitives to pixel fragments.

Clipping

Points, line segments, and polygons are handled slightly differently during clipping. Points are either retained in their original state (if they're inside the clip volume) or discarded (if they're outside). If portions of line segments or polygons are outside the clip volume, new vertices are generated at the clip points. For polygons, an entire edge may need to be constructed between such new vertices. For both line segments and polygons that are clipped, the edge flag, color, and texture information is assigned to all new vertices.

Clipping actually happens in two steps:

1. Application-specific clipping—Immediately after primitives are assembled, they're clipped in eye coordinates as necessary for any arbitrary clipping planes you've defined for your application with **glClipPlane**. (OpenGL requires support for at least six such application-specific clipping planes.)
2. View volume clipping—Next, primitives are transformed by the projection matrix (into clip coordinates) and clipped by the corresponding viewing volume. This matrix can be controlled by the previously mentioned matrix transformation commands but is most typically specified by **glFrustum** or **glOrtho**.

Transforming to Window Coordinates

Before clip coordinates can be converted to window coordinates, they are normalized by dividing by the value of *w* to yield normalized device coordinates. After that, the viewport transformation applied to these normalized coordinates produces window coordinates. You control the viewport, which determines the area of the on-screen window that displays an image, with **glDepthRange** and **glViewport**.

Rasterization

Rasterization is the process by which a primitive is converted to a two-dimensional image. Each point of this image contains such information as color, depth, and texture data. Together, a point and its associated information are called a fragment. The current raster position (as

specified with **glRasterPos**) is used in various ways during this stage for pixel drawing and bitmaps. As discussed below, different issues arise when rasterizing the three different types of primitives. In addition, pixel rectangles and bitmaps need to be rasterized.

Primitives. Control how primitives are rasterized with commands that allow you to choose dimensions and stipple patterns: **glPointSize**, **glLineWidth**, **glLineStipple**, and **glPolygonStipple**. In addition, you can control how the front and back faces of polygons are rasterized with **glCullFace**, **glFrontFace**, and **glPolygonMode**.

Pixels. Several commands control pixel storage and transfer modes. The command **glPixelStore** controls the encoding of pixels in client memory, and **glPixelTransfer** and **glPixelMap** control how pixels are processed before being placed in the frame buffer. A pixel rectangle is specified with **glDrawPixels**; its rasterization is controlled with **glPixelZoom**.

Bitmaps. Bitmaps are rectangles of zeros and ones specifying a particular pattern of fragments to be produced. Each of these fragments has the same associated data. A bitmap is specified using **glBitmap**.

Texture Memory. Texturing maps a portion of a specified texture image onto each primitive when texturing is enabled. This mapping is accomplished by using the color of the texture image at the location indicated by a fragment's texture coordinates to modify the fragment's RGBA color. A texture image is specified using **glTexImage2D** or **glTexImage1D**. The commands **glTexParameter** and **glTexEnv** control how texture values are interpreted and applied to a fragment.

Fog. You can have OpenGL blend a fog color with a rasterized fragment's post-texturing color using a blending factor that depends on the distance between the eyepoint and the fragment. Use **glFog** to specify the fog color and blending factor.

Primitives Reference

glBitmap
glClipPlane
glCullFace
glDepthRange
glDrawPixels
glFog
glFrontFace
glFrustum
glLineStipple
glLineWidth
glOrtho
glPixelMap
glPixelStore
glPixelTransfer
glPixelZoom
glPointSize
glPolygonMode
glPolygonStipple
glRasterPos
glTexEnv
glTexImage1D
glTexImage2D
glTexParameter
glViewport

Fragments

OpenGL allows a fragment produced by rasterization to modify the corresponding pixel in the

frame buffer only if it passes a series of tests. If it does pass, the fragment's data can be used directly to replace the existing frame buffer values, or it can be combined with existing data in the frame buffer, depending on the state of certain modes.

Pixel Ownership Test

The first test is to determine whether the pixel in the frame buffer corresponding to a particular fragment is owned by the current OpenGL context. If so, the fragment proceeds to the next test. If not, the window system determines whether the fragment is discarded or whether any further fragment operations will be performed with that fragment. This test allows the window system to control OpenGL's behavior when, for example, an OpenGL window is obscured.

Scissor Test

With the **glScissor** command, you can specify an arbitrary screen-aligned rectangle outside of which fragments will be discarded.

Alpha Test

The alpha test (which is performed only in RGBA mode) discards a fragment depending on the outcome of a comparison between the fragment's alpha value and a constant reference value. The comparison command and reference value are specified with **glAlphaFunc**.

Stencil Test

The stencil test conditionally discards a fragment based on the outcome of a comparison between the value in the stencil buffer and a reference value. The command **glStencilFunc** specifies the comparison command and the reference value. Whether the fragment passes or fails the stencil test, the value in the stencil buffer is modified according to the instructions specified with **glStencilOp**.

Depth Buffer Test

The depth buffer test discards a fragment if a depth comparison fails; **glDepthFunc** specifies the comparison command. The result of the depth comparison also affects the stencil buffer update value if stenciling is enabled.

Blending

Blending combines a fragment's R, G, B, and A values with those stored in the frame buffer at the corresponding location. The blending, which is performed only in RGBA mode, depends on the alpha value of the fragment and that of the corresponding currently stored pixel; it might also depend on the RGB values. You control blending with **glBlendFunc**, which allows you to indicate the source and destination blending factors.

Dithering

If dithering is enabled, a dithering algorithm is applied to the fragment's color or color index value. This algorithm depends only on the fragment's value and its *x* and *y* window coordinates.

Logical Operations

Finally, a logical operation can be applied between the fragment and the value stored at the corresponding location in the frame buffer; the result replaces the current frame buffer value. You choose the desired logical operation with **glLogicOp**. Logical operations are performed only on color indices, never on RGBA values.

Fragments Reference

glAlphaFunc
glBlendFunc

glDepthFunc
glLogicOp
glScissor
glStencilFunc
glStencilOp

Pixels

During the previous stage of the OpenGL pipeline, fragments are converted to pixels in the frame buffer. The frame buffer is actually organized into a set of logical buffers—the color, depth, stencil, and accumulation buffers. The color buffer itself consists of a front left, front right, back left, back right, and some number of auxiliary buffers. You can issue commands to control these buffers, and you can directly read or copy pixels from them. (Note that the particular OpenGL context you're using may not provide all of these buffers.)

Frame Buffer Operations

You can select into which buffer color values are written with **glDrawBuffer**. In addition, four different commands are used to mask the writing of bits to each of the logical frame buffers after all per-fragment operations have been performed: **glIndexMask**, **glColorMask**, **glDepthMask**, and **glStencilMask**. The operation of the accumulation buffer is controlled with **glAccum**. Finally, **glClear** sets every pixel in a specified subset of the buffers to the value specified with **glClearColor**, **glClearIndex**, **glClearDepth**, **glClearStencil**, or **glClearAccum**.

Reading or Copying Pixels

You can read pixels from the frame buffer into memory, encode them in various ways, and store the encoded result in memory with **glReadPixels**. In addition, you can copy a rectangle of pixel values from one region of the frame buffer to another with **glCopyPixels**. The command **glReadBuffer** controls from which color buffer the pixels are read or copied.

Pixels Reference

glAccum
glClear
glClearAccum
glClearColor
glClearDepth
glClearIndex
glClearStencil
glColorMask
glCopyPixels
glDepthMask
glDrawBuffer
glIndexMask
glReadBuffer
glReadPixels
glStencilMask

Using Evaluators

OpenGL's evaluator commands allow you to use a polynomial mapping to produce vertices, normals, texture coordinates, and colors. These calculated values are then passed on to the pipeline as if they had been directly specified. The evaluator facility is also the basis for the

NURBS (Non-Uniform Rational B-Spline) commands, which allow you to define curves and surfaces, as described in **OpenGL Utility Library**.

The first step involved in using evaluators is to define the appropriate one- or two-dimensional polynomial mapping using **glMap**. The domain values for this map can then be specified and evaluated in one of two ways:

- By defining a series of evenly spaced domain values to be mapped using **glMapGrid** and then evaluating a rectangular subset of that grid with **glEvalMesh**. A single point of the grid can be evaluated using **glEvalPoint**.
- By explicitly specifying a desired domain value as an argument to , which evaluates the maps at that value.

Evaluators Reference

glEvalCoord
glEvalMesh
glEvalPoint
glMap
glMapGrid

Performing Selection and Feedback

Selection, feedback, and rendering are mutually exclusive modes of operation. Rendering is the normal, default mode during which fragments are produced by rasterization. In selection and feedback modes, no fragments are produced; therefore, no frame buffer modification occurs. In selection mode, you can determine which primitives would be drawn into some region of a window; in feedback mode, information about primitives that would be rasterized is fed back to the application. You select among these three modes with **glRenderMode**.

Selection

Selection works by returning the current contents of the name stack, which is an array of integer-valued names. You assign the names and build the name stack within the modeling code that specifies the geometry of objects you want to draw. Then, in selection mode, whenever a primitive intersects the clip volume, a selection hit occurs. The hit record, which is written into the selection array you've supplied with **glSelectBuffer**, contains information about the contents of the name stack at the time of the hit. (Note that **glSelectBuffer** needs to be called before OpenGL is put into selection mode with **glRenderMode**. Also, the entire contents of the name stack isn't guaranteed to be returned until **glRenderMode** is called to take OpenGL out of selection mode.) You manipulate the name stack with **glInitNames**, **glLoadName**, **glPushName**, and **glPopName**. In addition, you might want to use an OpenGL Utility Library routine for selection, **gluPickMatrix**, which is described in **OpenGL Utility Library**.

Feedback

In feedback mode, each primitive that would be rasterized generates a block of values that is copied into the feedback array. You supply this array with **glFeedbackBuffer**, which must be called before OpenGL is put into feedback mode. Each block of values begins with a code indicating the primitive type, followed by values that describe the primitive's vertices and

associated data. Entries are also written for bitmaps and pixel rectangles. Values are not guaranteed to be written into the feedback array until **glRenderMode** is called to take OpenGL out of feedback mode. You can use **glPassThrough** to supply a marker that's returned in feedback mode as if it were a primitive.

Selection and Feedback Reference

glRenderMode

Selection

glInitNames
glLoadName
gluPickMatrix
glPopName
glPushName
glSelectBuffer

Feedback

glFeedbackBuffer
glPassThrough

Using Display Lists

A display list is simply a group of OpenGL commands that has been stored for subsequent execution. The **glNewList** command begins the creation of a display list, and **glEndList** ends it. With few exceptions, OpenGL commands called between **glNewList** and **glEndList** are appended to the display list, and optionally executed as well. (**glNewList** lists the commands that can't be stored and executed from within a display list.) To trigger the execution of a list or set of lists, use **glCallList** or **glCallLists** and supply the identifying number of a particular list or lists. You can manage the indices used to identify display lists with **glGenLists**, **glListBase**, and **glIsList**. Finally, you can delete a set of display lists with **glDeleteLists**.

Display Lists Reference

glCallList
glCallLists
glDeleteLists
glEndList
glGenLists
glIsList
glListBase
glNewList

Managing Modes and Execution

The effect of many OpenGL commands depends on whether a particular mode is in effect. You use **glEnable** and **glDisable** to set such modes and **glIsEnabled** to determine whether a

particular mode is set.

You can control the execution of previously issued OpenGL commands with **glFinish**, which forces all such commands to complete, or **glFlush**, which ensures that all such commands will be completed in a finite time.

A particular implementation of OpenGL may allow certain behaviors to be controlled with hints, by using the **glHint** command. Possible behaviors are the quality of color and texture coordinate interpolation, the accuracy of fog calculations, and the sampling quality of antialiased points, lines, or polygons.

Modes and Execution Reference

glDisable
glEnable
glFinish
glFlush
glHint
glIsEnabled

Obtaining State Information

OpenGL maintains many state variables that affect the behavior of many commands. Some of these variables have specialized query commands:

glGetLight	**glGetTexEnv**	**glGetTexParameter**
glGetMaterial	**glGetTexGen**	**glGetMap**
glGetClipPlane	**glGetTexImage**	**glGetPixelMap**
glGetPolygonStipple	**glGetTexLevelParameter**	

The value of other state variables can be obtained with **glGetBooleanv**, **glGetDoublev**, **glGetFloatv**, or **glGetIntegerv**, as appropriate. See **glGet** for information about how to use these commands. Other query commands you might want to use are **glGetError**, **glGetString**, and **glIsEnabled**. (See **Handling Errors** for more information about routines related to error handling.) Finally, you can save and restore sets of state variables with **glPushAttrib** and **glPopAttrib**.

The Query Commands

There are four commands for obtaining simple state variables, and one for determining whether a particular state is enabled or disabled.

void **glGetBooleanv**(GLenum *pname*, GLboolean **params*);void **glGetIntegerv**(GLenum *pname*, GLint **params*);void **glGetFloatv**(GLenum *pname*, GLfloat **params*);void **glGetDoublev**(GLenum *pname*, GLdouble **params*);

Obtains Boolean, integer, floating-point, or double-precision state variables. The *pname* argument is a symbolic constant indicating the state variable to return, and *params* is a pointer to an array of the indicated type in which to place the returned data. The possible values for *pname* are listed in **OpenGL State Variables**. A type conversion is performed if necessary to

return the desired variable as the requested data type.

GLboolean **glIsEnabled**(GLenum *cap*);

Returns GL_TRUE if the mode specified by *cap* is enabled; otherwise, returns GL_FALSE. The possible values for *cap* are listed in **OpenGL State Variables**.

Other specialized commands return specific state variables. The prototypes for these commands are listed here. To find out when to use these commands, see **OpenGL State Variables**. Also see the *OpenGL Reference Manual*. OpenGL's error handling facility and the **glGetError** command are described in more detail in **Error Handling**.

void **glGetClipPlane**(GLenum *plane*,GLdouble **equation*);GLenum **glGetError**(void);void **glGetLight**{if}**v**(GLenum *light*,GLenum *pname*, *TYPE *params*);void **glGetMap**{ifd}**v**(GLenum *target*,GLenum *query TYPE *v*);void **glGetMaterial**{if}**v**(GLenum *face*,GLenum *pname*, *TYPE *params*);void **glGetPixelMap**{if ui us}**v**(GLenum *map*, *TYPE *values*);void **glGetPolygonStipple**(GLubyte **mask*);void **glGetPolygonStipple**(GLubyte **mask*);const GLubyte ****glGetString**(GLenum *name*);void **glGetTexEnv**{if}**v**(GLenum *target*, GLenum *pname*, *TYPE *params*);void **glGetTexGen**{ifd}**v**(GLenum *coord,* GLenum *pname*, *TYPE *params*);void **glGetTexImage**(GLenum *target*, GLint *level*, GLenum *format,* GLenum *type,* GLvoid **pixels*);void **glGetTexLevelParameter**{if}**v**(GLenum *target*, GLint *level*, GLenum *pname*, *TYPE *params*);void **glGetTexParameter**{if}**v**(GLenum *target*,GLenum *pname*, *TYPE *params*);

Error Handling

When OpenGL detects an error, it records a current error code. The command that caused the error is ignored, so it has no effect on OpenGL state or on the frame-buffer contents. (If the error recorded was GL_OUT_OF_MEMORY, however, the results of the command are undefined.) Once recorded, the current error code isn't cleared until you call the query command **glGetError**, which returns the current error code.

Distributed implementations of OpenGL may return multiple current error codes, each of which remains set until queried. The **glGetError** function returns GL_NO_ERROR once you've queried all the current error codes, or if there's no error, so if you obtain an error code, it's a good practice to call **glGetError** until GL_NO_ERROR is returned to be sure you've discovered all the errors. See **OpenGL error codes**.

You can use the GLU routine **gluErrorString** to obtain a descriptive string corresponding to the error code passed in. This routine is described in more detail in **Describing Errors**. Notice that GLU routines often return error values if an error is detected. Also, the GLU defines the error codes GLU_INVALID_ENUM, GLU_INVALID_VALUE, and GLU_OUT_OF_MEMORY, which

have the same meaning as the related OpenGL codes.

OpenGL Error Codes

Error Code	Description
GL_INVALID_ENUM	GLenum argument out of range
GL_INVALID_VALUE	Numeric argument out of range
GL_INVALID_OPERATION	Operation illegal in current state
GL_STACK_OVERFLOW	Command would cause a stack overflow
GL_STACK_UNDERFLOW	Command would cause a stack underflow
GL_OUT_OF_MEMORY	Not enough memory left to execute command

Saving and Restoring Sets of State Variables

You can save and restore the values of a collection of state variables on an attribute stack with the commands **glPushAttrib** and **glPopAttrib**. The attribute stack has a depth of at least 16, and the actual depth can be obtained using GL_MAX_ATTRIB_STACK_DEPTH with **glGetIntegerv**. Pushing a full stack or popping an empty one generates an error.

In general, it's faster to use **glPushAttrib** and **glPopAttrib** than to get and restore the values yourself. Some values might be pushed and popped in the hardware, and saving and restoring them might be expensive. Also, if you're operating on a remote client, all the attribute data must be transferred across the network connection and back as it's saved and restored. However, your OpenGL implementation keeps the attribute stack on the server, avoiding unnecessary network delays.

void **glPushAttrib**(GLbitfield *mask*);

Saves all the attributes indicated by bits in *mask* by pushing them onto the attribute stack. The following Attribute Groups table lists the possible mask bits than can be logically ORed together to save any combination of attributes. Each bit corresponds to a collection of individual state variables. For example, GL_LIGHTING_BIT refers to all the state variables related to lighting, which include the current material color, the ambient, diffuse, specular, and emitted light, a list of the lights that are enabled, and the directions of the spotlights. When **glPopAttrib** is called, all those variables are restored. To find out exactly which attributes are saved for particular mask values, see OpenGL State Variables.

OpenGL State Variables

The following topics list the names of queryable state variables:

State Variables for Current Values and Associated Data

Transformation State Variables

Coloring State Variables

Lighting State Variables

Rasterization State Variables

Texturing State Variables

Pixel Operations

Framebuffer Control State Variables

Pixel State Variables

Evaluator State Variables

Hint State Variables

Implementation-Dependent State Variables

Implementation-Dependent Pixel-Depth State Variables

Miscellaneous State Variables

For each variable, the topic lists a description, attribute group, initial or minimum value, and the suggested **glGet*** command to use for obtaining it. State variables that can be obtained using **glGetBooleanv**, **glGetIntegerv**, **glGetFloatv**, or **glGetDoublev** are listed with just one of these commands — the one that's most appropriate give the type of data to be returned. These state variables can't be obtained using **glIsEnabled**. However, state variables for which **glIsEnabled** is listed as the query command can also be obtained using **glGetBooleanv( )**, **glGetIntegerv**, **glGetFloatv**, and **glGetDoublev**. State variables for which any other command is listed as the query command can be obtained only by using that command. If no attribute group is listed, the variable doesn't belong to any group. All queryable state variables except the implementation-dependent ones have initial values. If no initial value is listed, consult the topic where that variable is discussed to determine its initial value.

State Information Reference

For a list of the reference pages for the state variables, see OpenGL State Variables.

glGet
glGetBooleanv
glGetClipPlane
glGetDoublev
glGetError
glGetFloatv
glGetIntegerv
glIsEnabled
glGetLight
glGetMap
glGetMaterial
glGetPixelMap
glGetPolygonStipple
glGetString
glGetTexEnv
glGetTexGen
glGetTexImage
glGetTexLevelParameter
glGetTexParameter
glPopAttrib
glPushAttrib

OpenGL Utility Library

The OpenGL Utility Library (GLU) contains several groups of commands that complement the core OpenGL interface by providing support for auxiliary features. These utility routines make use of core OpenGL commands, so any OpenGL implementation is guaranteed to support the utility routines. Note that the prefix for Utility Library routines is "glu" rather than "gl."

- Consult the *OpenGL Reference Manual* for more detailed descriptions of these routines. This section groups them functionally as follows: • Initialization
- Manipulating Images for Use in Texturing
- Transforming Coordinates
- Tessellation
- Rendering Spheres, Cylinders, and Disks
- NURBS Curves and Surfaces
- Describing Errors

Initialization

With GLU version 1.1 or later you can use a routine that returns the version number of GLU library or that returns the version number and any vendor-specific GLU extension calls (**gluGetString**).

const GLubyte *__gluGetString__(GLenum *name*);

Manipulating Images for Use in Texturing

The OpenGL Utility Library (GLU) provides image scaling and automatic mipmapping routines to simplify the specification of texture images. The routine **gluScaleImage** scales a specified image to an accepted texture size; the resulting image can then be passed to OpenGL as a texture. The automatic mipmapping routines **gluBuild1DMipmaps** and **gluBuild2DMipmaps** create mipmapped texture images from a specified image and pass them to **glTexImage1D** and **glTexImage2D**, respectively.

As you set up texture mapping in your application, you'll probably want to take advantage of mipmapping, which requires a series of reduced images (or texture maps). To support mipmapping, the GLU includes a general routine that scales images (**gluScaleImage**) and routines that generate a complete set of mipmaps given an original image in one or two dimensions (**gluBuild1DMipmaps** and **gluBuild2DMipmaps**).

GLint **gluScaleImage(**GLenum *format***,** GLint *widthin***,** GLint *heightin***,**
GLenum *typein***,** const void ***__datain__**,
GLint *widthout***,** GLint *heightout***,**
GLenum *typeout***,** void ***__dataout__**);GLint **gluBuild1DMipmaps(**GLenum *target***,** GLint *components***,**
GLint *width***,** GLenum *format***,** GLenum *type***,**
void ***__data__**);GLint **gluBuild2DMipmaps(**GLenum *target***,** GLint *components***,**
GLint *width***,** GLint *height***,** GLenum *format***,**
GLenum *type***,** void ***__data__**);

Transforming Coordinates

The OpenGL Utility Library (GLU) provides several commonly used matrix transformation routines. You can set up a two-dimensional orthographic viewing region with **gluOrtho2D**, a perspective viewing volume using **gluPerspective**, or a viewing volume that is centered on a specified eyepoint with **gluLookAt**. Each of these routines creates the desired matrix and

applies it to the current matrix using **glMultMatrix**.

The **gluPickMatrix** routine simplifies selection by creating a matrix that restricts drawing to a small region of the viewport. If you rerender the scene in selection mode after this matrix has been applied, all objects that would be drawn near the cursor will be selected, and information about them will be stored in the selection buffer. See **Performing Selection and Feedback** for more information about selection mode.

To determine where in the window an object is being drawn, use the **gluProject** function, which converts specified coordinates from object coordinates to window coordinates. The **gluUnProject** function performs the inverse conversion.

The GLU includes routines that create matrices for standard perspective and orthographic viewing (**gluPerspective** and **gluOrtho2D**). In addition, a viewing routine allows you to position your eye at any point in space and look at any other point (**gluLookAt**). In addition, the GLU includes a routine to help you create a picking matrix (**gluPickMatrix**). The prototypes for these four routines are listed here.

void **gluPerspective(**GLdouble *fovy***,** GLdouble *aspect***,** GLdouble *zNear***,**
GLdouble *zFar***);**void **gluOrtho2D(**GLdouble *left***,** GLdouble *right***,** GLdouble *bottom***,**
GLdouble *top***);**void **gluLookAt(**GLdouble *eyex***,** GLdouble *eyey***,** GLdouble *eyez***,**
GLdouble *centerx***,** GLdouble *centery***,**
GLdouble *centerz***,** GLdouble *upx***,** GLdouble *upy***,**
GLdouble *upz*); void **gluPickMatrix(**GLdouble *x***,** GLdouble *y***,** GLdouble *width***,**
GLdouble *height***,** GLint *viewport*[4]);

In addition, GLU provides two routines that convert between object coordinates and screen coordinates, **gluProject** and **gluUnProject**.

GLint **gluProject(**GLdouble *objx***,** GLdouble *objy***,** GLdouble *objz***,**
const GLdouble *modelMatrix*[16]**,**
const GLdouble *projMatrix*[16]**,**
const GLint *viewport*[4]**,** GLdouble ***winx***,**
GLdouble ****winy***,** GLdouble ****winz*****);**

Transforms the specified object coordinates *objx*, *objy*, and *objz* into window coordinates using *modelMatrix*, *projMatrix*, and *viewport*. The result is stored in *winx*, *winy*, and *winz*. A return value of GL_TRUE indicates success, and GL_FALSE indicates failure.

GLint **gluUnProject(**GLdouble *winx***,** GLdouble *winy***,** GLdouble *winz***,**
const GLdouble *modelMatrix*[16]**,**
const GLdouble *projMatrix*[16]**,**
const GLint *viewport*[4]**,** GLdouble ***objx***,**
GLdouble ****objy***,** GLdouble ****objz*****);**

Transforms the specified window coordinates *winx*, *winy*, and *winz* into object coordinates using *modelMatrix*, *projMatrix*, and *viewport*. The result is stored in *objx*, *objy*, and *objz*. A return value of GL_TRUE indicates success, and GL_FALSE indicates failure.

Polygon Tessellation

The polygon tessellation routines triangulate a concave polygon with one or more contours. To use this GLU feature, first create a tessellation object with **gluNewTess**, and define callback routines that will be used to process the triangles generated by the tessellator (with **gluTessCallBack**). Then use **gluBeginPolygon**, **gluTessVertex**, **gluNextContour**, and **gluEndPolygon** to specify the concave polygon to be tessellated. Unneeded tessellation

objects can be destroyed with **gluDeleteTess**.

OpenGL can directly display only simple convex polygons. A polygon is simple if the edges intersect only at vertices, there are no duplicate vertices, and exactly two edges meet at any vertex. If your application requires the display of simple nonconvex polygons or of simple polygons containing holes, those polygons must first be subdivided into convex polygons before they can be displayed. Such subdivision is called *tessellation*. GLU provides a collection of routines that perform tessellation. Note that the GLU tessellation routines can't handle nonsimple polygons; there's no standard OpenGL method to handle such polygons.

Because tessellation is often required and can be rather tricky, this section describes the GLU tessellation routines in detail. These routines take as input arbitrary simple polygons that might include holes, and they return some combination of triangles, triangle meshes, and triangle fans. You can insist on triangles only if you don't want to have to deal with meshes or fans. If you care about performance, however, you should probably take advantage of any available mesh or fan information.

The Callback Mechanism

To tessellate a polygon using the GLU, first create a tessellation object, then provide a series of callback routines to be called at appropriate times during the tessellation. After specifying the callbacks, describe the polygon and any holes using GLU routines, which are similar to the OpenGL polygon routines. When the polygon description is complete, the tessellation facility invokes your callback routines as necessary.

The callback routines typically save the data for the triangles, triangle meshes, and triangle fans in user-defined data structures, or in OpenGL display lists. To render the polygons, other code traverses the data structures or calls the display lists. Although the callback routines could call OpenGL commands to display them directly, this is usually not done, as tessellation can be computationally expensive. It's a good idea to save the data if there is any chance that you want to display it again. The GLU tessellation routines are guaranteed never to return any new vertices, so interpolation of vertices, texture coordinates, or colors is never required.

The Tessellation Object

As a complex polygon is being described and tessellated, it has associated data, such as the vertices, edges, and callback functions. All this data is tied to a single tessellation object. To tessellate, your program must first create a tessellation object using the routine **gluNewTess**.

GLUtriangulatorObj*** gluNewTess(**void**);**

Creates a new tessellation object and returns a pointer to it. A null pointer is returned if the creation fails.

If you no longer need a tessellation object, you can delete it and free all associated memory with **gluDeleteTess**.

void **gluDeleteTess(**GLUtriangulatorObj ****tessobj*****);**

Deletes the specified tessellation object, *tessobj*, and frees all associated memory.

A single tessellation object can be reused for all your tessellations. This object is required only because library routines might be required to do their own tessellations, and they should be able to do so without interfering with any tessellation that your program is doing. It might also be useful to have multiple tessellation objects if you want to use different sets of callbacks for different tessellations. A typical program, however, allocates a single tessellation object and uses it for all its tessellations. There's no real need to free it because it uses a small amount of memory. On the other hand, if you're writing a library routine that uses the GLU tessellation, you'll want to be careful to free any tessellation objects you create.

Specifying Callbacks

You can specify up to five callback functions for a tessellation. Any functions that are omitted are simply not called during the tessellation, and any information they might have returned to your program is lost. All are specified by the single routine **gluTessCallback**.

void **gluTessCallback(**GLUtriangulatorObj ***tessobj*, GLenum *type*,
void **(*** *fn*)( ));

Associates the callback function fn with the tessellation object *tessobj*. The type of the callback is determined by the parameter *type*, which can be GLU_BEGIN, GLU_EDGE_FLAG, GLU_VERTEX, GLU_END, or GLU_ERROR. The five possible callback functions have the following prototypes:

GLU_BEGIN	void **begin(**GLenum *type***);**
GLU_EDGE_FLAG	void **edgeFlag(**GLboolean *flag***);**
GLU_VERTEX	void **vertex(**void ***** *data***);**
GLU_END	void **end(***void***);**
GLU_ERROR	void **error(**GLenum *errno***);**

To change a callback routine, simply call **gluTessCallback** with the new routine. To eliminate a callback routine without replacing it with a new one, pass **gluTessCallback** a null pointer for the appropriate function.

As tessellation proceeds, these routines are called in a manner similar to the way you would use the OpenGL commands **glBegin**, **glEdgeFlag**, **glVertex**, and **glEnd**. The error callback is invoked during the tessellation only if something goes wrong.

The GLU_BEGIN callback is invoked with one of three possible parameters: GL_TRIANGLE_FAN, GL_TRIANGLE_STRIP, or GL_TRIANGLES. After this routine is called, and before the callback associated with GLU_END is called, some combination of the GLU_EDGE_FLAG and GLU_VERTEX callbacks is invoked. The associated vertices and edge flags are interpreted exactly as they are in OpenGL between **glBegin(GL_TRIANGLE_FAN)**, **glBegin(GL_TRIANGLE_STRIP)**, or **glBegin(GL_TRIANGLES)** and the matching **glEnd**. Since edge flags make no sense in a triangle fan or triangle strip, if there is a callback associated with GLU_EDGE_FLAG, the GLU_BEGIN callback is called only with GL_TRIANGLES. The GLU_EDGE_FLAG callback works exactly analogously to the OpenGL **glEdgeFlag** call.

The error callback is passed a GLU error number. A character string describing the error can be obtained using the routine **gluErrorString**.

Describing the Polygon to Be Tessellated

The polygon to be tessellated, possibly containing holes, is specified using the following four routines: **gluBeginPolygon**, **gluTessVertex**, **gluNextContour**, and **gluEndPolygon**. For polygons without holes, the specification is exactly as in OpenGL: start with **gluBeginPolygon**, call **gluTessVertex** for each vertex in the boundary, and end the polygon with a call to **gluEndPolygon**. If a polygon consists of multiple contours, including holes and holes within holes, the contours are specified one after the other, each preceded by **gluNextContour**. When **gluEndPolygon** is called, it signals the end of the final contour and starts the tessellation. You can omit the call to **gluNextContour** before the first contour. The detailed descriptions of these functions follow.

void **gluBeginPolygon(**GLUtriangulatorObj ***tessobj*);

Begins the specification of a polygon to be tessellated and associates a tessellation object,

tessobj, with it. The callback functions to be used are those that were bound to the tessellation object using the routine **gluTessCallback**.

void **gluTessVertex(**GLUtriangulatorObj ***tessobj*,
GLdouble *v*[3], void ***** *data*);

Specifies a vertex in the polygon to be tessellated. Call this routine for each vertex in the polygon to be tessellated. *tessobj* is the tessellation object to use, *v* contains the three-dimensional vertex coordinates, and *data* is an arbitrary pointer that is sent to the callback associated with GLU_VERTEX. Typically, it contains vertex data, texture coordinates, color information, or whatever else the application may require.

void **gluNextContour(**GLUtriangulatorObj ***tessobj*, GLenum *type***);**

Marks the beginning of the next contour when multiple contours make up the boundary of the polygon to be tessellated. *type* can be GLU_EXTERIOR, GLU_INTERIOR, GLU_CCW, GLU_CW, or GLU_UNKNOWN. These serve only as hints to the tessellation. If you get them right, the tessellation might go faster. If you get them wrong, they're ignored, and the tessellation still works. For a polygon with holes, one contour is the exterior contour and the others interior. **gluNextContour** can be called immediately after **gluBeginPolygon**, but if it isn't, the first contour is assumed to be of type GLU_EXTERIOR. GLU_CW and GLU_CCW indicate clockwise- and counterclockwise- oriented polygons. Choosing which are clockwise and which are counterclockwise is arbitrary in three dimensions, but in any plane, there are two different orientations, and the GLU_CW and GLU_CCW types should be used consistently. Use GLU_UNKNOWN if you don't know which to use.

void **gluEndPolygon(**GLUtriangulatorObj ***tessobj*);

Indicates the end of the polygon specification and that the tessellation can begin using the tessellation object *tessobj*.

Rendering Simple Surfaces

You can render spheres, cylinders, and disks using the GLU quadric routines. To do this, create a quadric object with **gluNewQuadric**. (To destroy this object when you're finished with it, use **gluDeleteQuadric**.) Then specify the desired rendering style, as listed below, with the appropriate routine (unless you're satisfied with the default values):

- Whether surface normals should be generated, and if so, whether there should be one normal per vertex or one normal per face: **gluQuadricNormals**
- Whether texture coodinates should be generated: **gluQuadricTexture**
- Which side of the quadric should be considered the outside and which the inside: **gluQuadricOrientation**
- Whether the quadric should be drawn as a set of polygons, lines, or points: **gluQuadricDrawStyle**

After you've specified the rendering style, simply invoke the rendering routine for the desired type of quadric object: **gluSphere**, **gluCylinder**, **gluDisk**, or **gluPartialDisk**. If an error occurs during rendering, the error-handling routine you've specified with **gluQuadricCallBack** is invoked.

The GLU includes a set of routines for drawing various simple surfaces (spheres, cylinders, disks, and parts of disks) in a variety of styles and orientations. These routines are described in detail in the *OpenGL Reference Manual*; their use is discussed briefly in the following paragraphs, and their prototypes are also listed.

To create a quadric object, use **gluNewQuadric**. (To destroy this object when you're finished with it, use **gluDeleteQuadric**.) Then specify the desired rendering style, as follows, with the appropriate routine (unless you're satisfied with the default values):

- Whether surface normals should be generated, and if so, whether there should be one

normal per vertex or one normal per face: **gluQuadricNormals**.

- Whether texture coodinates should be generated: **gluQuadricTexture**.
- Which side of the quadric should be considered the outside and which the inside: **gluQuadricOrientation**.
- Whether the quadric should be drawn as a set of polygons, lines, or points: **gluQuadricDrawStyle**.

After you've specified the rendering style, simply invoke the rendering routine for the desired type of quadric object: **gluSphere**, **gluCylinder**, **gluDisk**, or **gluPartialDisk**. If an error occurs during rendering, the error-handling routine you've specified with **gluQuadricCallBack** is invoked.

It's better to use the ***Radius*, *height*, and similar arguments to scale the quadrics rather than the **glScale** command, so that unit-length normals that are generated don't have to be renormalized. Set the *loops* and *stacks* arguments to values other than 1 to force lighting calculations at a finer granularity, especially if the material specularity is high.

The prototypes are listed in three categories.

Manage quadric objects:

```
GLUquadricObj* gluNewQuadric (void);
    void gluDeleteQuadric (GLUquadricObj *state);
    void gluQuadricCallback (GLUquadricObj *qobj, GLenum which,
     void (*fn)( ));
```

Control the rendering:

```
void gluQuadricNormals (GLUquadricObj *quadObject,
     GLenum normals);
    void gluQuadricTexture (GLUquadricObj *quadObject,
     GLboolean textureCoords);
    void gluQuadricOrientation (GLUquadricObj *quadObject,
     GLenum orientation);
    void gluQuadricDrawStyle (GLUquadricObj *quadObject,
     GLenum drawStyle);
```

Specify a quadric primitive:

```
void gluCylinder (GLUquadricObj *qobj, GLdouble baseRadius,
     GLdouble topRadius, GLdouble height, GLint slices,
     GLint stacks);
    void gluDisk (GLUquadricObj *qobj, GLdouble innerRadius,
     GLdouble outerRadius, GLint slices, GLint loops);
    void gluPartialDisk (GLUquadricObj *qobj, GLdouble innerRadius,
     GLdouble outerRadius, GLint slices, GLint loops,
     GLdouble startAngle, GLdouble sweepAngle);
    void gluSphere (GLUquadricObj *qobj, GLdouble radius, GLint slices,
     GLint stacks);
```

NURBS Curves and Surfaces

Non-Uniform Rational B-Spline (NURBS) curves and surfaces are converted to OpenGL evaluators by the routines described in this section. You can create and delete a NURBS object with **gluNewNurbsRenderer** and **gluDeleteNurbsRenderer**, and establish an error-handling routine with **gluNurbsCallback**.

You specify the desired curves and surfaces with different sets of routines — **gluBeginCurve**, **gluNurbsCurve**, and **gluEndCurve** for curves or **gluBeginSurface**, **gluNurbsSurface**, and **gluEndSurface** for surfaces. You can also specify a trimming region, which defines a subset of

the NURBS surface domain to be evaluated, thereby allowing you to create surfaces that have smooth boundaries or that contain holes. The trimming routines are **gluBeginTrim**, **gluPwlCurve**, **gluNurbsCurve**, and **gluEndTrim**.

As with quadric objects, you can control how NURBS curves and surfaces are rendered:

- Whether a curve or surface should be discarded if its control polyhedron lies outside the current viewport
- What the maximum length should be (in pixels) of edges of polygons used to render curves and surfaces
- Whether the projection matrix, modelview matrix, and viewport should be taken from the OpenGL server or whether you'll supply them explictly with **gluLoadSamplingMatrices**.

Use **gluNurbsProperty** to set these properties, or use the default values. You can query a NURBS object about its rendering style with **gluGetNurbsProperty**.

NURBS routines provide general and powerful descriptions of curves and surfaces in two and three dimensions. They're used to represent geometry in many computer-aided mechanical design systems. The GLU NURBS routines can render such curves and surfaces in a variety of styles, and they can automatically handle adaptive subdivision that tessellates the domain into smaller triangles in regions of high curvature and near silhouette edges.

Manage a NURBS object:

```
GLUnurbsObj* gluNewNurbsRenderer (void);
    void gluDeleteNurbsRenderer (GLUnurbsObj *nobj);
    void gluNurbsCallback (GLUnurbsObj *nobj, GLenum which,
     void (*fn)( ));
```

Create a NURBS curve:

```
void gluBeginCurve (GLUnurbsObj *nobj);
    void gluEndCurve (GLUnurbsObj *nobj);
    void gluNurbsCurve (GLUnurbsObj *nobj, GLint nknots, GLfloat *knot,
     GLint stride, GLfloat *ctlarray,
     GLint order, GLenum type);
```

Create a NURBS surface:

```
void gluBeginSurface (GLUnurbsObj *nobj);
    void gluEndSurface (GLUnurbsObj *nobj);
    void gluNurbsSurface (GLUnurbsObj *nobj, GLint uknot_count,
     GLfloat *uknot, GLint vknot_count, GLfloat *vknot,
     GLint u_stride, GLint v_stride, GLfloat *ctlarray,
     GLint uorder, GLint vorder, GLenum type);
```

Define a trimming region:

```
void gluBeginTrim (GLUnurbsObj *nobj);
    void gluEndTrim (GLUnurbsObj *nobj);
    void gluPwlCurve (GLUnurbsObj *nobj, GLint count, GLfloat *array,
     GLint stride, GLenum type);
```

Control NURBS rendering:

```
void gluLoadSamplingMatrices (GLUnurbsObj *nobj,
     const GLfloat modelMatrix[16],
     const GLfloat projMatrix[16],
     const GLint viewport[4]);
    void gluNurbsProperty (GLUnurbsObj *nobj, GLenum property,
     GLfloat value);
    void gluGetNurbsProperty (GLUnurbsObj *nobj, GLenum property,
     GLfloat *value);
```

Handling Errors

The routine **gluErrorString** is provided for retrieving an error string that corresponds to an OpenGL or GLU error code. The currently defined OpenGL error codes are described in **glGetError**. The GLU error codes are listed in **gluErrorString**, **gluTessCallback**, **gluQuadricCallback**, and **gluNurbsCallback**.

The GLU provides a routine for obtaining a descriptive string for an error code.

const GLubyte***** **gluErrorString(**GLenum *errorCode*)**;**

Returns a pointer to a descriptive string that corresponds to the OpenGL, GLU, or GLX error number passed in *errorCode*. The defined error codes are described in the *OpenGL Reference Manual* along with the command or routine that can generate them.